

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

KITA KOD introduces an AI-powered demand intelligence solution for Malaysian SMEs, especially mini markets and kedai runcit operators. The system addresses fragmented business data and manual decision-making by combining internal records with external market signals and routing them through Z.AI GLM for explainable recommendations.

1.1. Purpose:

This system analysis document explains the technical scope, architecture, workflow, and AI integration decisions behind KITA KOD. It serves as a reference for developers, testers, and evaluators who need to understand how the product is designed and how Z.AI GLM functions as the reasoning core of the system.

So, the key element of system that has been covered are as followed:

1. Architecture:

- a. KITA KOD follows a client-server web architecture. A React + Vite frontend interacts with a Flask backend over REST APIs, while the backend coordinates business logic, mock/external signal preparation, and Z.AI GLM calls.

2. Data Flows:

- a. Business data flows from user input forms or CSV uploads into the frontend state, then into backend endpoints for analysis, forecasting, action generation, and AI assistance. Responses are returned as JSON and rendered in the dashboard pages.

3. Model Process:

- a. The model process follows a layered Predict -> Recommend -> Explain pattern. GLM receives structured context, reasons over the business state, and produces forecast or recommendation outputs in a consumable format.

4. Role of Reference:

- a. This document is a technical reference for the team to align architecture, API behavior, external dependency assumptions, and future implementation work.

1.2. Background:

KITA KOD is designed for SMEs that usually operate with isolated data and intuition-based decisions. Instead of requiring advanced analytics skill, the platform organizes business inputs and contextual signals into a guided AI-assisted workflow for forecasting and action planning.

1. The current project is the initial prototype for UMHackathon 2026. There is no previous production version; the system is being introduced as a new web-based decision intelligence platform.
2. Changes in Major Architectural Components
3. New capabilities introduced in this version include a dedicated GLM service layer, tiered business data input, multi-page decision dashboards, and explicit support for external signal display such as trends, holidays, and weather.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
SME Owner	Primary end user who enters business data and reviews forecasts and actions	Simple, reliable, and explainable recommendations that improve decisions
Development Team	Builds and integrates frontend, backend, and GLM service logic	Modular code, clean service boundaries, and clear API contracts
QA Team	Validates prototype behavior, edge cases, and output reliability	Testable flows, predictable output structure, and low-friction defect tracing
Hackathon Judges	Evaluators of product thinking, architecture, and execution	Clear GLM centrality, strong reasoning value, and credible technical design
Team Members	Internal collaborators across product, engineering, and presentation	A shared technical reference for implementation and pitching

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web App (React frontend + Flask backend)
Architecture	Client-server web architecture with dedicated GLM service layer

KITA KOD is primarily structured as a web-based client-server system. The React frontend provides business data entry, analysis views, and AI assistant pages. The Flask backend exposes `/api/\*` endpoints, prepares prompt context, and communicates with Z.AI GLM through a dedicated service layer. The current prototype uses in-memory frontend state and mock-backed backend business outputs where full integration is still pending.

2.1.2 LLM as Service Layer

The architecture uses Z.AI GLM as a service layer for reasoning, not as a generic chatbot module. The backend is responsible for constructing prompt context, calling the GLM endpoint, and parsing returned responses into frontend-safe payloads.

2.1.3 Dependency Diagram

Component	Description
User Interface	Captures sales data and user queries.
Backend API	Normalizes data and constructs context-rich prompts.
Z.AI GLM	Processes prompts (Context + Data + Query) and generates structured insights.
Response Parser	Extracts JSON recommendations from GLM's natural language output.
Action Module	Displays concrete tasks (e.g., "Restock Item A") to the user.
Database	Persistent storage for historical prompts and generated insights for consistency.

Questions Answered:

- Prompt Construction: Prompts are built dynamically by concatenating historical sales CSV data, current weather forecasts, and search trends into a structured "System Context".
- Context Window: We utilize a rolling window of the last 3 months of sales data to stay within the 8,000 token limit.
- Parsing: The `parse_json_response` utility in `glm_service.py` uses regex and JSON parsing to ensure structured data is returned to the UI.

2.1.4 Sequence Diagram

Step	Actor	System	Description
1	User	UI	Clicks "Generate Prediction" on Dashboard.
2	UI	Backend	Sends request with timeframe parameters.
3	Backend	Database	Retrieves last 30 days of sales/inventory data.
4	Backend	External	Fetches upcoming weather and holiday calendar.
5	Backend	Z.AI GLM	Sends aggregated context: "Predict demand given this data..."
6	Z.AI GLM	Backend	Returns reasoning + forecast numbers.
7	Backend	UI	Returns formatted JSON to the frontend.
8	UI	User	Displays chart with "Why" explanation.

2.2. Technological Stack

Layer	Technology	Justification
Frontend	React (TypeScript)	Robust component-based architecture for complex dashboards.
Styling	Vanilla CSS	High performance and full control over interactive feedback/animations.
Backend	Flask (Python)	Lightweight and excellent for AI/ML integration.
Database	SQLite	Zero-configuration, file-based database ideal for SME local deployments.
AI/ML	Z.AI GLM	Central reasoning engine for decision intelligence.

2.3. Key Data Flows

This section shows how data moves through KITA KOD, how it is structured, and how it is returned to the user after processing.

Level	Description
External Entities	SME Owners, Google Trends API, OpenWeather API.
Processes	Data Ingestion, Trend Correlation, GLM Reasoning, Action Mapping.

Level	Description
Data Stores	kitakod.db (Relational), Local Cache (Session).
Data Flows	CSV → Backend → DB; DB + External → GLM → UI.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	Business Data Input	Users enter or upload sales, expenses, products, inventory, and related SME business data through tiered forms.
2	Demand Forecasting	The system generates near-term demand predictions using historical patterns plus contextual signals.
3	Action Suggestions	AI-generated recommendations turn business insights into practical next steps such as restocking or promotions.
4	AI Assistant / What-if	Users ask natural-language business questions and receive explainable trade-off analysis through Z.AI GLM.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirements	Implementation
Scalability	The structure should support more modules, business types, and external data sources without major rewrites.	Frontend pages and backend services are already separated into modular files for future expansion.
Reliability	Core forecasting and AI response flows must remain usable even when external data is incomplete or unavailable.	Mock fallback behavior and health checks are already present in the backend and frontend status flow.
Maintainability	The codebase should remain understandable for rapid hackathon iteration and later extension.	Routes, services, components, pages, and context are separated clearly across frontend and backend folders.

4. API Documentation Reference

Endpoint	Method	Request	Response	Description
/api/forecast	POST	{days: int}	{forecast: [], trend: str}	Generates sales predictions.
/api/ai/ask	POST	{question: str}	{answer: str}	Conversational interface for data queries.
/api/external/fetch	POST	{sources: []}	{trends: [], weather: []}	Fetches real-time market signals.

4.1. External Dependencies:

Tools	Purpose	Risks
Z.AI GLM API	Core reasoning engine for context-aware forecasting, recommendations, and AI what-if analysis	High - if the GLM API is unavailable, the product loses its strongest decision-support value and falls back to mock behavior.
Browser Fetch / REST APIs	Communication layer between frontend pages and backend endpoints	Medium - broken API contracts or invalid payloads will block major user flows.
External Signal Sources	Weather, holiday, and trend-like contextual inputs displayed or simulated in the prototype	Medium - real integration is not complete yet, so credibility depends on clear labeling of current prototype scope.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline: The development timeline below summarizes the team progression from domain understanding to final submission in a sprint-like sequence.

Workstream	16 Apr	17 Apr	18 Apr	19 Apr	20 Apr	21 Apr	22 Apr	23 Apr	24 Apr	25 Apr	26 Apr
Domain Understanding	ACTI VE	ACTI VE	ACTI VE								

[illegible]

Email: umhackathon@um.edu.my

19-21 Apr 2026	Planning & Setup	Defined scope, architecture, project flow, and development setup.	Working frontend/backend foundation.
22 Apr 2026	AI Integration Preparation	Obtained the API key and began validating the Z.AI GLM testing flow.	AI integration path prepared.
23-24 Apr 2026	Core Development	Built the main pages, backend endpoints, and prototype logic for forecasting and actions.	Functional MVP modules.
25 Apr 2026	Refinement & QA	Improved usability, checked flows, and finalized documentation and pitch material.	Stable demo flow and submission assets.
26 Apr 2026	Submission	Final review and hand-in of the project package.	Completed submission.

5.2. Team Members Role: The following roles were assigned to ensure clear ownership across planning, design, implementation, and validation.

Team Member	Role	Responsibilities
Ng Ern Yi	Project Manager	Managed project direction, coordinated tasks, aligned deliverables with judging criteria, and oversaw final submission.
Chew Jia Hui	Frontend Developer	Developed the React frontend, dashboard pages, and user-facing workflow.
Tey Henry	Backend Developer	Built the Flask backend, API routes, and GLM integration flow.
Ooi Wei Shen	UI/UX Designer	Designed the interface, improved clarity, and shaped the product presentation quality.
Wong Wen Hao	QA	Tested user flows, checked edge cases, and supported final validation before submission.

5.3. Recommendations: Based on the current prototype state, the following recommendations are proposed to strengthen scalability, performance, and reliability.

Area	Recommendation	Purpose
AI Integration	Increase the role of Z.AI GLM in forecast, actions, and what-if features so the AI is visibly central to the product.	Strengthen judging performance for reasoning and decision intelligence.

External Data	Replace more mock signals with live or semi-live external data where feasible.	Improve credibility and realism during demo.
Persistence	Add localStorage or lightweight backend persistence for key business data.	Improve reliability and prevent loss of entered data.
Testing	Expand negative, edge-case, and prompt-boundary testing coverage.	Improve QA confidence and technical defensibility.
Scalability	Keep the architecture flexible enough to support other SME sectors later.	Show future product potential beyond the current mini-market focus.
Documentation Consistency	Align pitch, PRD, SAD, and testing documents to the same product story and actual implementation state.	Reduce inconsistency during judging and Q&A.